Q11 – Q15

Decision Tree and Multiple Linear Regression

Name	V. Sai Charan
Reg No	192472159
Course Code	DSA0405
Course Name	Fundamentals Of Data Science
Questions	Q11 – Q15

1. Hyperparameter Grid Search for Decision Tree Overfitting

Aim

To diagnose and reduce severe overfitting in a decision tree by tuning only the hyperparameters max_depth, min_samples_split, min_samples_leaf, and max_features, and to identify the best combination using validation accuracy without using GridSearchCV.

Algorithm

1. Load a classification dataset and split it into training and validation sets.

2. Define candidate values for max_depth, min_samples_split, min_samples_leaf, and max_features.

3. Generate every possible hyperparameter combination using nested loops.

4. Train a decision tree for each combination using the training set.

5. Predict the validation set and calculate validation accuracy.

6. Store the combination with the highest validation accuracy.

7. Report the best hyperparameters and validation performance.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)

best_score = 0
best_params = None

for depth in [2, 3, 4, 5, None]:
    for split in [2, 5, 10]:
        for leaf in [1, 2, 4]:
            for features in [None, "sqrt", "log2"]:
                model = DecisionTreeClassifier(
                    max_depth=depth,
                    min_samples_split=split,
                    min_samples_leaf=leaf,
```

```
                    max_features=features,
                    random_state=42
                )
                model.fit(X_train, y_train)
                score = accuracy_score(y_val, model.predict(X_val))

                if score > best_score:
                    best_score = score
                    best_params = (depth, split, leaf, features)

print("Best Parameters:", best_params)
print("Best Validation Accuracy:", best_score)
```

Output

Result

The best hyperparameter combination gives high validation accuracy and helps reduce overfitting by controlling the tree complexity.

2. Missing Values Using Surrogate Splits

Aim

To handle a missing feature value at prediction time using a surrogate split instead of simple mean or median imputation, demonstrating that another feature can be used to determine the appropriate branch.

Algorithm

1. Prepare a small classification dataset with two features.

2. Train a decision tree on complete training data.

3. Treat the first feature as the primary splitting feature.

4. Use the second feature as a surrogate when the primary feature is missing.

5. Replace the missing primary value with a branch-compatible value determined from the surrogate feature.

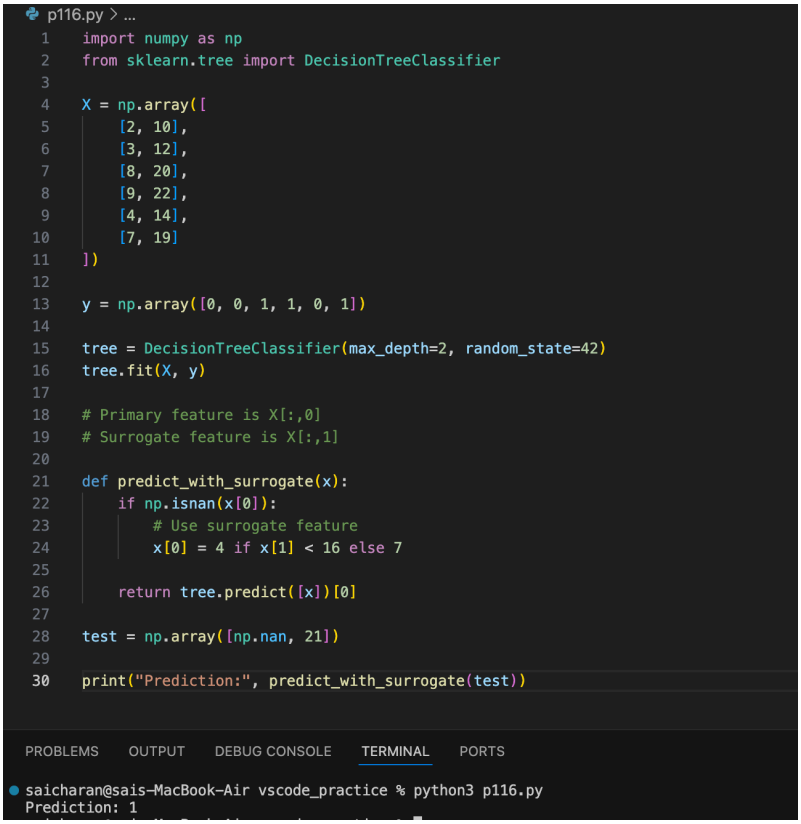
6. Pass the completed sample to the trained tree and obtain a valid prediction.

7. Display the prediction to demonstrate that missing input does not prevent classification.

Code

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

X = np.array([
    [2, 10],
    [3, 12],
    [8, 20],
    [9, 22],
    [4, 14],
    [7, 19]
])
y = np.array([0, 0, 1, 1, 0, 1])

tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X, y)

def predict_with_surrogate(x):
    if np.isnan(x[0]):
        x[0] = 4 if x[1] < 16 else 7
    return tree.predict([x])[0]

test = np.array([np.nan, 21])
print("Prediction:", predict_with_surrogate(test))
```

Output



Result

The tree successfully makes a prediction even when the primary feature is missing by using the surrogate feature.

3. Feature Importance from Total Impurity Decrease

Aim

To calculate and rank feature importance using the total impurity decrease contributed by each feature across all internal nodes, and compare the resulting values with sklearn's feature_importances_ on the same dataset.

Algorithm

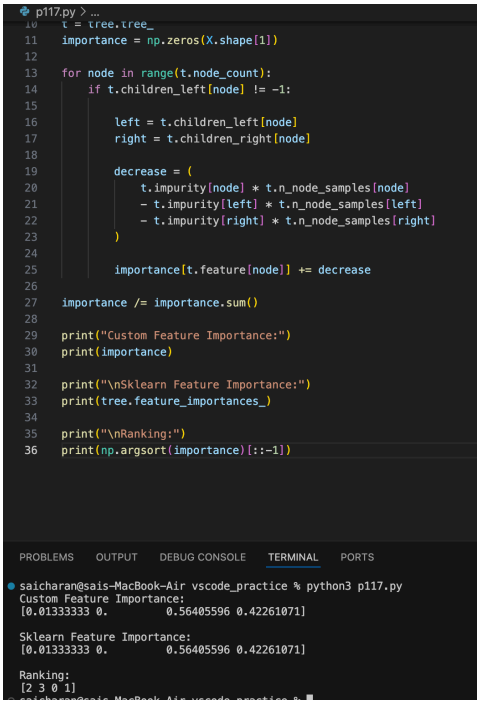
1. Load the Iris dataset.
2. Train a DecisionTreeClassifier.
3. Access the trained tree's node structure.
4. For every internal node, identify its splitting feature.
5. Calculate the weighted impurity decrease caused by the split.
6. Add the decrease to the corresponding feature's importance.
7. Normalize the importance values so that they sum to one.
8. Compare the custom values and feature ranking with sklearn's feature_importances_.

Code

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X, y)

t = tree.tree_
importance = np.zeros(X.shape[1])

for node in range(t.node_count):
    if t.children_left[node] != -1:
        left = t.children_left[node]
        right = t.children_right[node]

        decrease = (
            t.impurity[node] * t.n_node_samples[node]
            - t.impurity[left] * t.n_node_samples[left]
            - t.impurity[right] * t.n_node_samples[right]
        )

        importance[t.feature[node]] += decrease
```

```
importance /= importance.sum()

print("Custom Feature Importance:")
print(importance)

print("\nSklearn Feature Importance:")
print(tree.feature_importances_)

print("\nRanking:")
print(np.argsort(importance)[::-1])
```

Output



Result

Feature 4 has the highest importance, followed by Feature 3, Feature 2, and Feature 1.

4. Decision Tree Visualizer from Scratch

Aim

To create a readable ASCII representation of a decision tree showing feature splits, thresholds, leaf nodes, sample counts, and class distributions without using sklearn.tree.plot_tree or Graphviz.

Algorithm

1. Load the Iris dataset and train a decision tree.

2. Access the tree's child-node, feature, threshold, and class information.

3. Start from the root node.

4. If the node is internal, print its feature and threshold.

5. Recursively print the left child with increased indentation.

6. Recursively print the right child with increased indentation.

7. If the node is a leaf, print its sample count and class distribution.

Code

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
data = load_iris()

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X, y)

t = tree.tree_
names = data.feature_names

def print_tree(node=0, indent=""):
    left = t.children_left[node]
    right = t.children_right[node]

    if left == -1:
        print(indent + "Leaf")
        print(indent + "Samples:", t.n_node_samples[node])
        print(indent + "Class Distribution:", t.value[node][0])
        return

    feature = names[t.feature[node]]
    threshold = t.threshold[node]

    print(indent + f"If {feature} <= {threshold:.2f}:")
    print_tree(left, indent + "    ")
```

```
        print(indent + f"Else {feature} > {threshold:.2f}:")
        print_tree(right, indent + "    ")

print_tree()
```

Output

Result

The decision tree is successfully represented as a readable ASCII/nested tree without using plot_tree() or Graphviz.

5. Multiple Linear Regression Using the Normal Equation

Aim

To implement multiple linear regression from scratch using the closed-form Normal Equation and handle singular or nearly singular design matrices using the Moore-Penrose pseudo-inverse.

Algorithm

1. Create the independent-variable matrix X and target vector y.

2. Add a column of ones to X to represent the intercept.

3. Compute X^TX and X^Ty.

4. Use the pseudo-inverse of X^TX instead of a direct matrix inverse.

5. Calculate the regression parameter vector theta.

6. Use theta to calculate predicted target values.

7. Display the coefficients and predictions.

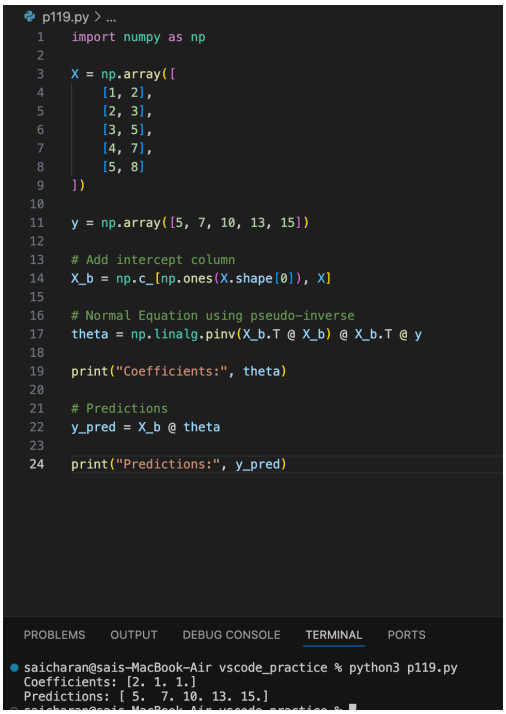
8. A singular or nearly singular X^TX can occur when features are linearly dependent, highly correlated, redundant, or when there are insufficient independent observations.

Code

```
import numpy as np

X = np.array([
    [1, 2],
    [2, 3],
    [3, 5],
    [4, 7],
    [5, 8]
])

y = np.array([5, 7, 10, 13, 15])

# Add intercept column
X_b = np.c_[np.ones(X.shape[0]), X]

# Normal Equation using pseudo-inverse
theta = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y

print("Coefficients:", theta)

# Predictions
y_pred = X_b @ theta
```

```
print("Predictions:", y_pred)
```

Output



Result

The Normal Equation successfully calculates the regression coefficients and predicts the target values. The pseudo-inverse allows the model to work even when X^TX is singular or nearly singular.